Our Accelerator, Explained from Scratch

One small example, followed from start to finish. Convention: spike trains stream along the rows (left to right), running sums move down the columns. Internal working notes, 15 October 2026.

Six words used throughout. Weight: one number of a filter. **Spike train**: what one input neuron does over the T = 4 timesteps, written as 4 bits (1 = spike), e.g. 0011. **Running sum**: 4 numbers (one per timestep) that collect the weights for one output. **Label**: a small tag saying which filter or which pixel something belongs to. **Group**: pixels that travel through the array together. **Tile**: the part of a layer that fits on the array at once.

1. The problem

A spiking layer is simple arithmetic: for every output and every timestep, add up the weights of the inputs that spiked. It is cheap (additions only), but most of the work is wasted on zeros: after pruning, most weights are 0, and most spike bits are 0. Many accelerators also process the T timesteps one after another. Our design does three things on a simple grid of processing elements (PEs):

- **All timesteps at once:** each PE handles all T bits of a spike train in one cycle (Section 3).
- **Pack the weights:** filters that never need the same PE share one column, so zero weights take no PE (Section 4).
- **Pack the spike trains:** pixels whose spikes never collide travel together, so zero bits take fewer steps (Section 5).

Small labels keep everything separable, so the output can always be put back exactly (Sections 6 and 7). No PE ever has to compare or search; all grouping is decided before data enters the array.

2. The example and the correct answer

Input: 3x3, one channel, T = 4 (bits t0 t1 t2 t3)

1000	0000	0010
0100	0011	0000
0000	1000	0100

Filter A (2x2)

2	0
0	1

Filter B (2x2)

0	3
1	0

Stride 1, no padding: the 2x2 window fits in 4 places -> 4 output pixels

p0	p1
p2	p3

Each filter gives one output channel: output = 2 channels x 4 pixels x 4 timesteps.

Figure 1. Input: a 3x3 spike map with T = 4. Two 2x2 filters, A and B. The filter fits in 4 places, so each filter produces 4 output pixels, p0 to p3.

Flattened spikes S[k][p]: what each output pixel sees at each filter position

	p0	p1	p2	p3	Weights W[f][k] A	B
k0 (top-left)	1000	0000	0100	0011	2	0
k1 (top-right)	0000	0010	0011	0000	0	3
k2 (bottom-left)	0100	0011	0000	1000	0	1
k3 (bottom-right)	0011	0000	1000	0100	1	0

Row k = one position inside the 2x2 window; column p = one output pixel. Example: p3 sees x11 = 0011 at its top-left, so S[k0][p3] = 0011.

Figure 2. For each output pixel p, the spike train it sees at each of the 4 window positions k0 to k3 (top-left, top-right, bottom-left, bottom-right), and each filter's weight at that position.

Normal GEMM: Output[f][p] = sum over k of W[f][k] x S[k][p]

	k0	k1	k2	k3
A	2	0	0	1
B	0	3	1	0

x

	p0	p1	p2	p3
k0	1000	0000	0100	0011
k1	0000	0010	0011	0000
k2	0100	0011	0000	1000
k3	0011	0000	1000	0100

=

	p0	p1	p2	p3
A	[2,0,1,1]	[0,0,0,0]	[1,2,0,0]	[0,1,2,2]
B	[0,1,0,0]	[0,0,4,1]	[0,0,3,3]	[1,0,0,0]

"w x 0011" means: add w at every timestep whose bit is 1. Each output entry is a current with one value per timestep.

```
A, p0 = 2x1000 + 1x0011            = [2,0,1,1]
A, p1 = (no weight meets a spike)  = [0,0,0,0]
A, p2 = 2x0100 + 1x1000            = [1,2,0,0]
A, p3 = 2x0011 + 1x0100            = [0,1,2,2]
B, p0 = 1x0100                     = [0,1,0,0]
B, p1 = 3x0010 + 1x0011            = [0,0,4,1]
B, p2 = 3x0011                     = [0,0,3,3]
B, p3 = 1x1000                     = [1,0,0,0]
```

Figure 3. The answer. "2 x 0011" means: add 2 at every timestep whose bit is 1. Everything that follows must reproduce these 8 running sums.

3. The basic array (no packing yet)

The array is a grid of PEs: **one row per window position** (input k0..k3) and **one column per filter**. Before the layer starts, each PE is loaded with one weight, and that weight never moves (this is called **weight-stationary**). Pixels are then fed in one after another. A pixel's 4 spike trains enter at the left end of the 4 rows (spike train k into row k) and move **right**, so both filters see them. For every (filter, pixel) a running sum starts at 0 at the top of the filter's column, moves **down**, picks up one weight in each row, and leaves at the bottom when it is finished.

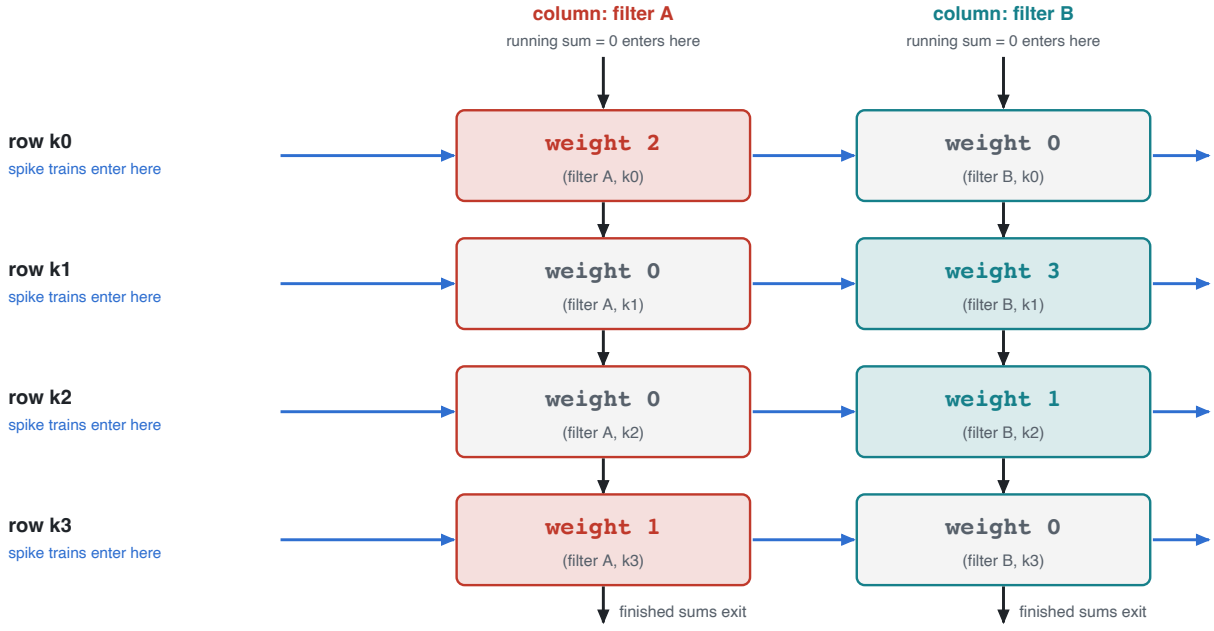


Figure 4. What stays and what moves. Weights never move. Spike trains move RIGHT along their row, one PE per cycle. Running sums move DOWN their column, one PE per cycle.

All timesteps at once. Each PE has 4 small adders, one per timestep. When a spike train arrives, every adder whose bit is 1 adds the PE's weight to that timestep of the running sum, all in the same cycle.

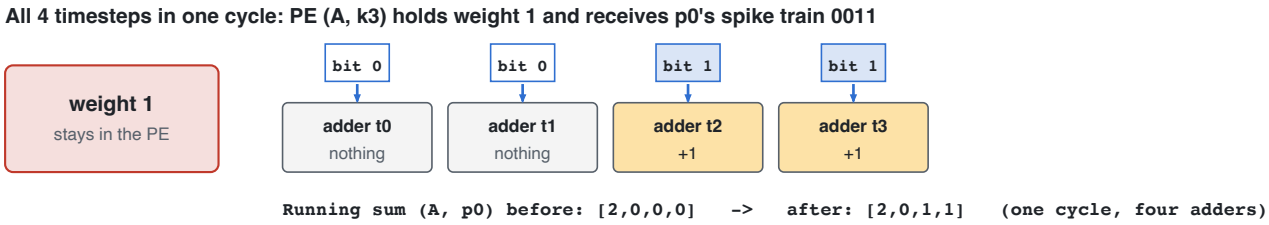


Figure 5. One PE, one cycle: the spike train 0011 makes adders t2 and t3 add the weight.

Cycle by cycle. Row k receives pixel n at cycle n + k, so pixels enter as a diagonal wave and many PEs work at the same time. Highlighted PEs are busy; each shows which pixel it is handling and the running sum after its addition.

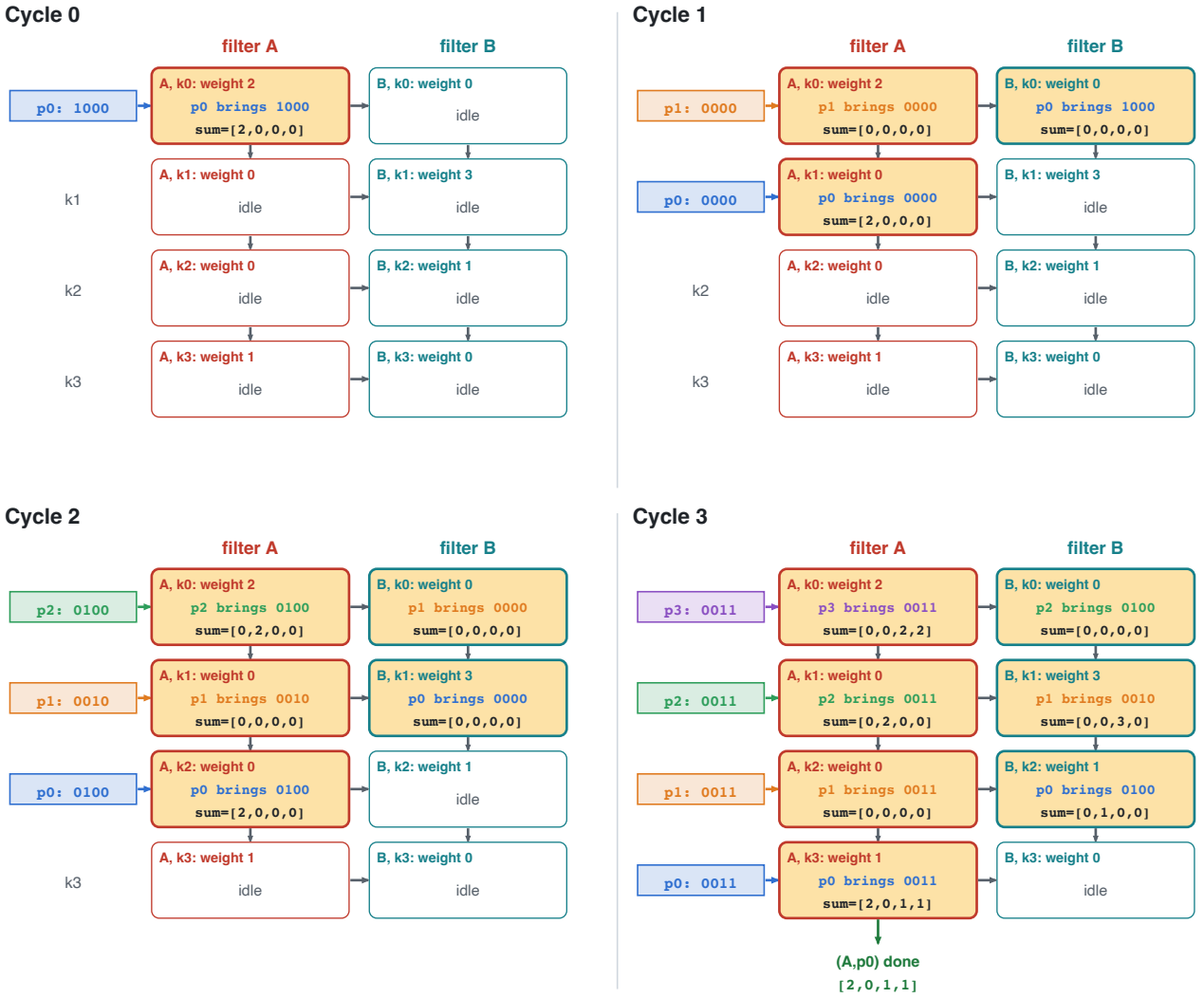


Figure 6. Cycles 0 to 3. In cycle 3 the first result, (A, p0) = [2,0,1,1], is finished while later pixels are still flowing behind it.

The whole schedule (which pixel each PE handles in each cycle):

PE (row, column) \ cycle	0	1	2	3	4	5	6	7
k0, A (weight 2)	p0	p1	p2	p3				
k0, B (weight 0)		p0	p1	p2	p3			
k1, A (weight 0)		p0	p1	p2	p3			
k1, B (weight 3)			p0	p1	p2	p3		
k2, A (weight 0)			p0	p1	p2	p3		
k2, B (weight 1)				p0	p1	p2	p3	
k3, A (weight 1)				p0	p1	p2	p3	
k3, B (weight 0)					p0	p1	p2	p3

What is wasted: 4 of the 8 PEs hold weight 0 and never add anything, and the 4 pixels need 4 streaming steps even though many of their spike bits are 0. Sections 4 and 5 remove these two wastes.

4. Idea 1: pack the weights

4.1 Deciding the packing (offline, once). Look at the weights row by row. Two filters can share one column if, in every row, at most one of them has a non-zero weight; that filter's weight goes into the PE, together with a label saying which filter it belongs to. In our example every row has exactly one non-zero weight, so A and B fit in a single column:

Row (input)	Weight of A	Weight of B	Non-zero owner	What the PE holds
k0	2	0	A	weight 2, label A
k1	0	3	B	weight 3, label B
k2	0	1	B	weight 1, label B
k3	1	0	A	weight 1, label A

If two filters both had a non-zero weight in the same row, they could not share that PE; the second filter would simply get its own column. Nothing is approximated: every non-zero weight is kept.

The packed column: 4 PEs instead of 8, every one holding a real weight

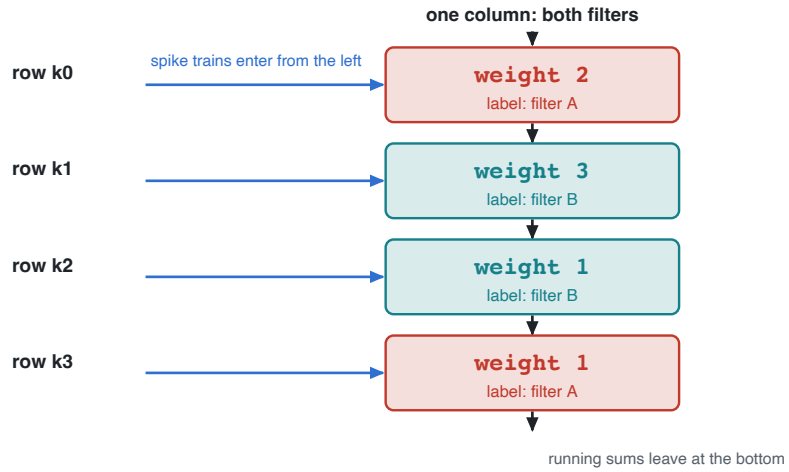


Figure 7. Two filters in one column. Spike trains still enter from the left, one per row.

4.2 What stays and what moves. Exactly as in Section 3: weights stay, spike trains enter from the left (row k receives pixel n at cycle n + k), and running sums move down. The one difference is that the column now serves both filters, so each pixel carries a **pair** of running sums down the column, one for A and one for B. They start at 0 at the top and travel together.

4.3 One PE, one cycle. The PE adds its weight exactly as before (one adder per timestep), but the filter label decides **which** of the two running sums receives the additions:

PE k2 (weight 1, label B) in cycle 2: pixel p0 arrives with spike train 0100

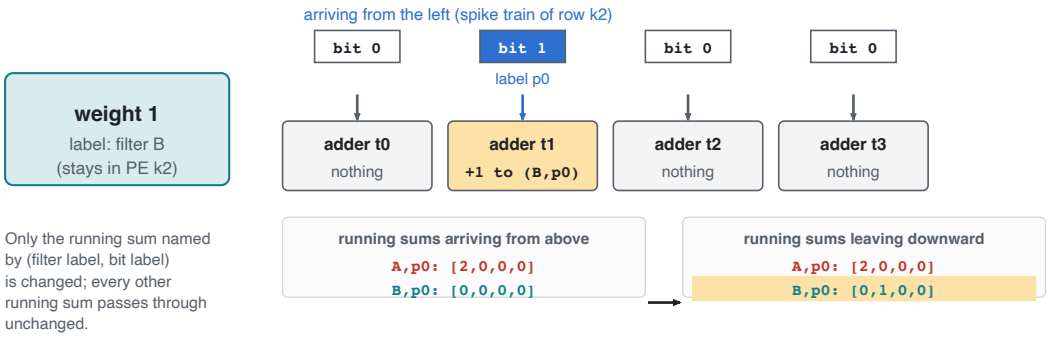


Figure 8. Bit t1 is 1, so adder t1 adds 1; the label B sends it to the running sum (B, p0). The sum (A, p0) passes down unchanged.

4.4 Cycle by cycle. Each snapshot shows the packed column. Left of a PE: the spike bits arriving in that row. Inside: what the PE adds ("p0@t1" = pixel p0, timestep t1). Right: the pair of running sums after the PE (the changed one highlighted), which then moves down to the next row.

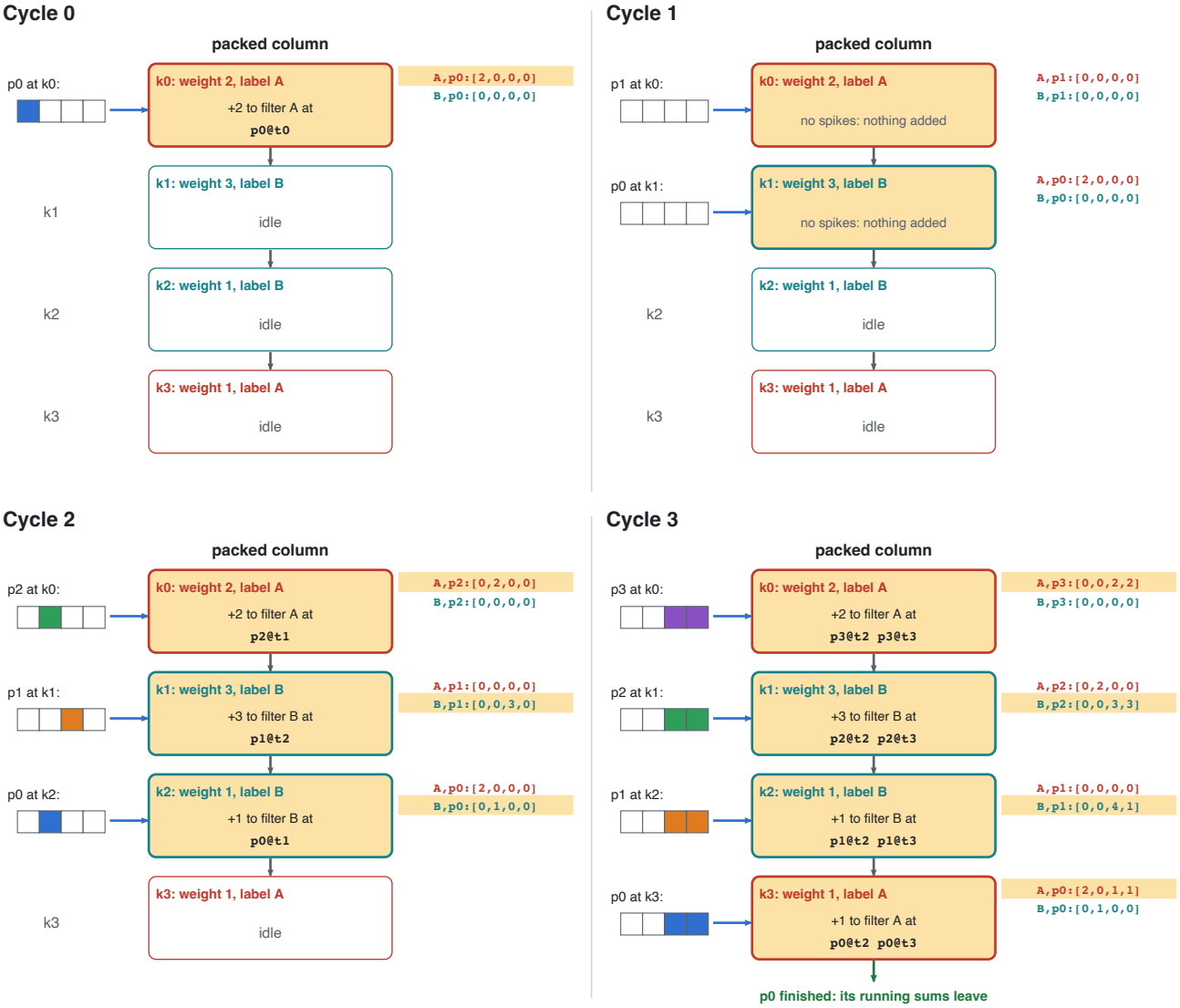


Figure 9. Cycles 0 to 3. In cycle 3, pixel p0 leaves the bottom with (A, p0) = [2,0,1,1] and (B, p0) = [0,1,0,0]; p1, p2 and p3 follow one cycle apart and leave after cycles 4, 5 and 6.

4.5 The whole schedule (which pixel each PE handles in each cycle):

PE (row) \ cycle	0	1	2	3	4	5	6
k0 (weight 2, label A)	p0	p1	p2	p3			
k1 (weight 3, label B)		p0	p1	p2	p3		
k2 (weight 1, label B)			p0	p1	p2	p3	
k3 (weight 1, label A)				p0	p1	p2	p3

4.6 What changed. The 4 PEs that only held zeros are gone, and every PE now adds a real weight. The number of streaming steps is unchanged (still one pixel per cycle), so the time saving here is small (the array is shorter, so it fills and drains faster). The gain is in hardware: half the PEs do the same work. The price: a label on every weight, and two running sums travelling down instead of one.

5. Idea 2: pack the spike trains

5.1 Why some pixels can travel together. A PE has exactly one adder per timestep. If two pixels arrive in a row at the same time, each adder can serve only one of them in that cycle. So two pixels can share a cycle if, **in every row, they never have a spike at the same timestep**. Check p0 and p1:

Row	p0	p1	Same timestep?
k0	1000	0000	no
k1	0000	0010	no
k2	0100	0011	no: p0 spikes at t1, p1 at t2 and t3
k3	0011	0000	no

5.2 Forming the groups (at run time). Spikes are only known when the previous layer produces them, so a small unit called the **packer** forms the groups just before they enter the array. It takes the pixels in order and puts each one into the first group it fits (at most 2 pixels per group here):

Step	Pixel	Check	Result
1	p0	no group yet	opens G1 = {p0}
2	p1	against G1: no shared timestep in any row	joins G1 = {p0, p1}
3	p2	against G1: clashes with p1 at row k1, timestep t2 (and G1 is full)	opens G2 = {p2}
4	p3	against G2: no shared timestep in any row	joins G2 = {p2, p3}

5.3 What enters each row. A group enters row k as one combined spike train (T bits), plus a **label for every bit** saying which pixel it came from ("-" = no spike):

Row	G1 bits	G1 labels (t0..t3)	G2 bits	G2 labels (t0..t3)
k0	1000	p0, -, -, -	0111	-, p2, p3, p3
k1	0010	-, -, p1, -	0011	-, -, p2, p2
k2	0111	-, p0, p1, p1	1000	p3, -, -, -
k3	0011	-, -, p0, p0	1100	p2, p3, -, -

A group carries **four** running sums down the column: (A or B) x (its two pixels). They start at 0 and travel together.

5.4 One PE, one cycle. Now two labels decide where each addition goes: the PE's filter label and the bit's pixel label.

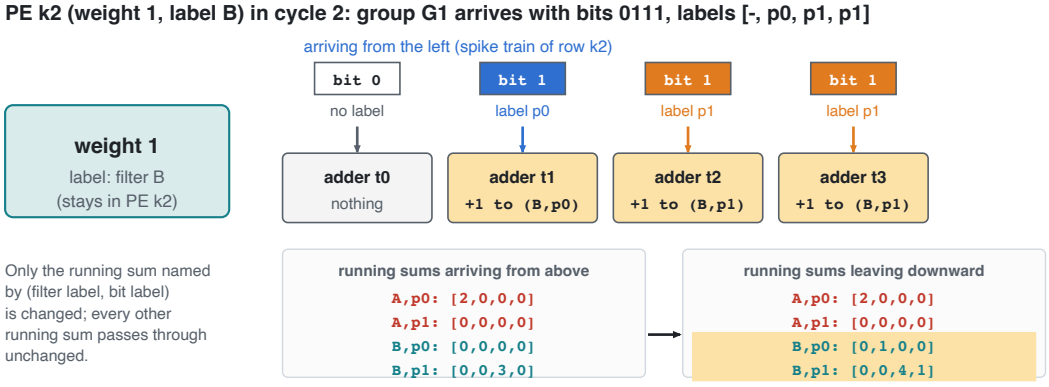


Figure 10. In one cycle, adder t1 adds 1 to (B, p0) and adders t2, t3 add 1 to (B, p1). Each adder serves exactly one pixel, which is what the clash check in 5.1 guarantees.

5.5 Cycle by cycle. Groups enter one per cycle (row k receives group g at cycle g + k). Left of each PE: the arriving bits, coloured by the pixel that owns them. Right: the four running sums after the PE.

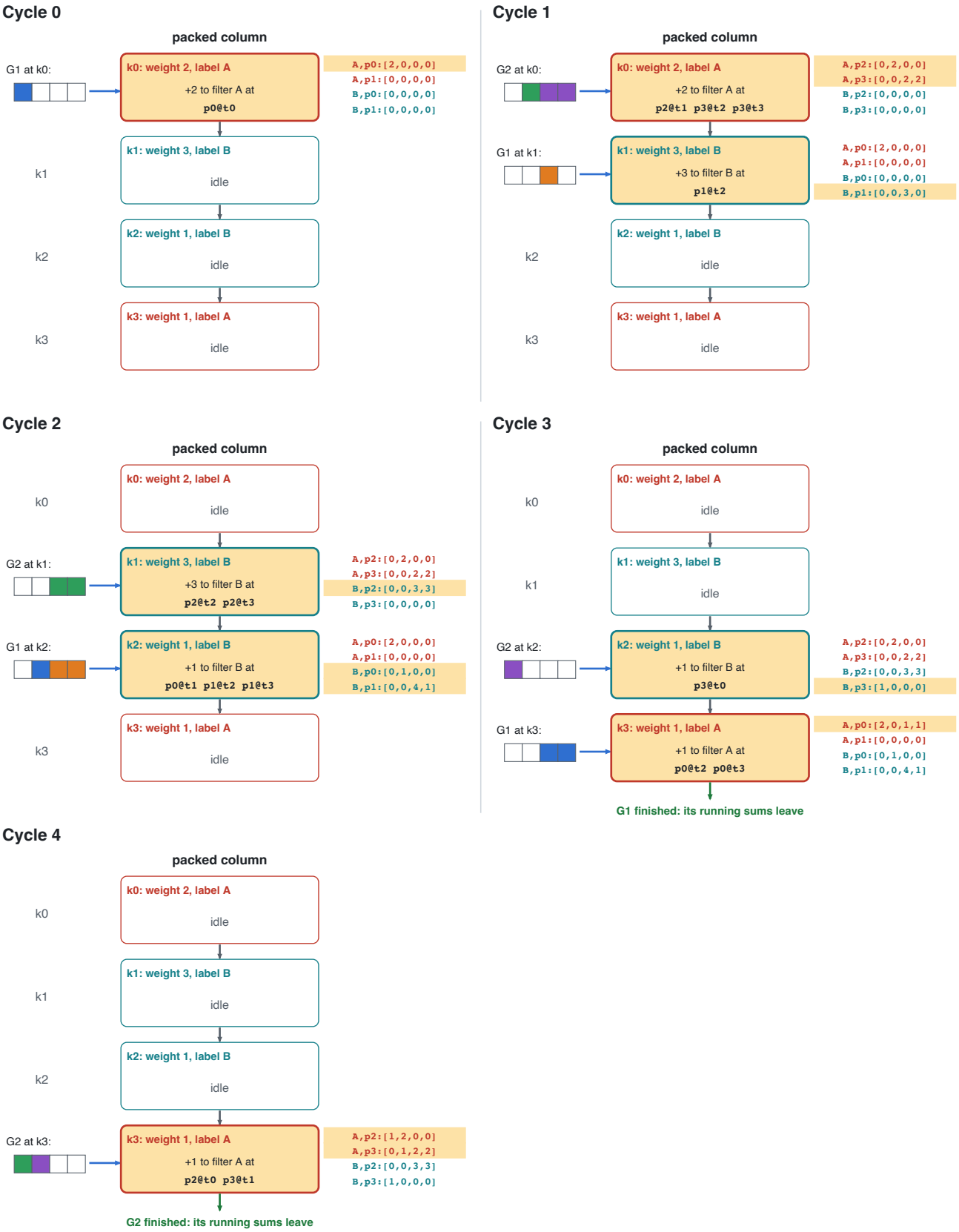


Figure 11. Cycles 0 to 4. G1 leaves the bottom after cycle 3 and G2 after cycle 4, each carrying four finished running sums.

5.6 The whole schedule (which group each PE handles in each cycle):

PE (row) \ cycle	0	1	2	3	4
k0 (weight 2, label A)	G1	G2			
k1 (weight 3, label B)		G1	G2		
k2 (weight 1, label B)			G1	G2	
k3 (weight 1, label A)				G1	G2

5.7 What changed.

	Basic array (Sec. 3)	+ weight packing (Sec. 4)	+ spike packing (Sec. 5)
PEs	8 (half hold 0)	4	4
Things streamed in	4 pixels	4 pixels	2 groups
Cycles until the last result leaves	8	7	5
Running sums travelling together	1	2 (A, B)	4 (A, B x 2 pixels)
Labels	none	filter label on each weight	+ pixel label on each spike bit

In a real layer with hundreds of pixels, the fill and drain cycles are negligible and the time is set by the number of things streamed in: weight packing reduces the PEs needed, and spike packing reduces the streaming steps (here by half, on real data by about 1.1-1.4x).

6. Putting the output back

When a group's running sums leave the bottom of the column, each already carries two labels: the filter (from the weight labels) and the pixel (from the spike labels). A unit below the array simply writes each running sum to output[filter][pixel]. No search is needed.

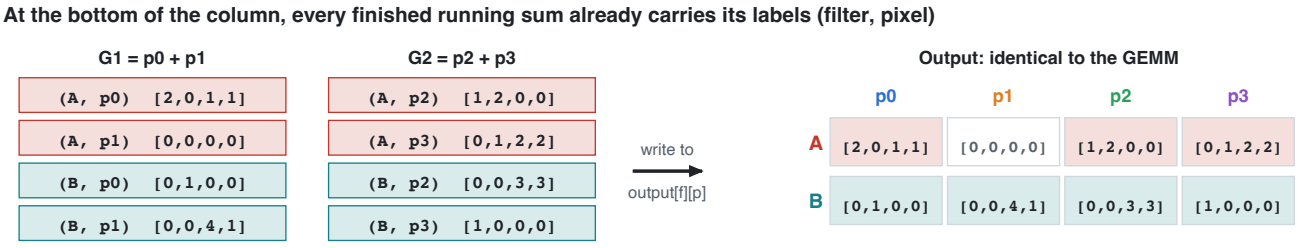


Figure 12. The two groups' running sums, written back into the normal output layout.

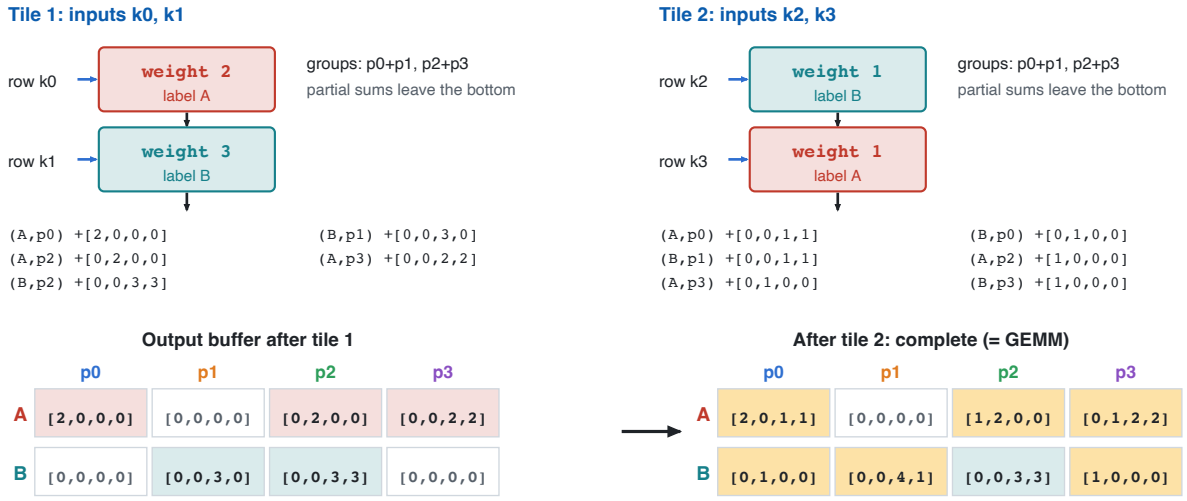
7. Bigger layers: tiles

Real layers have hundreds of inputs but the array has only a few rows, so a layer runs in pieces called tiles: each tile covers as many inputs as there are rows. Two things follow:

- Packing is decided **per tile**: which filters share a column and which pixels form a group may be different in each tile, because only that tile's weights and spikes matter.
- A running sum that leaves the bottom of the column after one tile is only **partial**. The edge unit adds it into an **output buffer** at (filter, pixel). After the last tile, every entry of the buffer is complete.

Here is our layer run on an array with only 2 rows. Tile 1 covers k0 and k1, tile 2 covers k2 and k3:

The same layer on an array with only 2 rows: two tiles, results added in the output buffer



Highlighted = changed by tile 2. Zero partial sums are skipped (nothing is written for them).

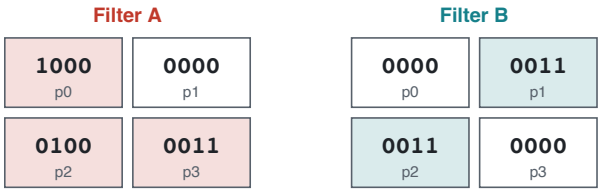
Figure 13. Tile 1 leaves partial sums in the buffer; tile 2 adds to them. In this small example the groups happen to be the same in both tiles; in a real layer they usually differ, and the labels still put every partial sum in the right place.

8. The neuron (LIF)

Only after the last tile is each running sum complete. The LIF unit then processes its 4 values in time order: V = 0.5 x V + value at t; if V >= 2 the neuron fires (spike = 1) and V drops by 2. This step has to go in time order, but it runs once per output, outside the array.

Output	Running sum	V at t0, t1, t2, t3 (* = fires)	Spike train out
A, p0	[2,0,1,1]	2*, 0, 1, 1.5	1000
A, p1	[0,0,0,0]	0, 0, 0, 0	0000
A, p2	[1,2,0,0]	1, 2.5*, 0.25, 0.125	0100
A, p3	[0,1,2,2]	0, 1, 2.5*, 2.25*	0011
B, p0	[0,1,0,0]	0, 1, 0.5, 0.25	0000
B, p1	[0,0,4,1]	0, 0, 4*, 2*	0011
B, p2	[0,0,3,3]	0, 0, 3*, 3.5*	0011
B, p3	[1,0,0,0]	1, 0.5, 0.25, 0.125	0000

Output spike maps (after LIF)



Two 2x2 spike maps, each pixel a 4-bit spike train: the input to the next layer.

Figure 14. The layer's output: one 2x2 spike map per filter. These spike trains are the input of the next layer, and the whole process repeats.

9. The hardware, block by block

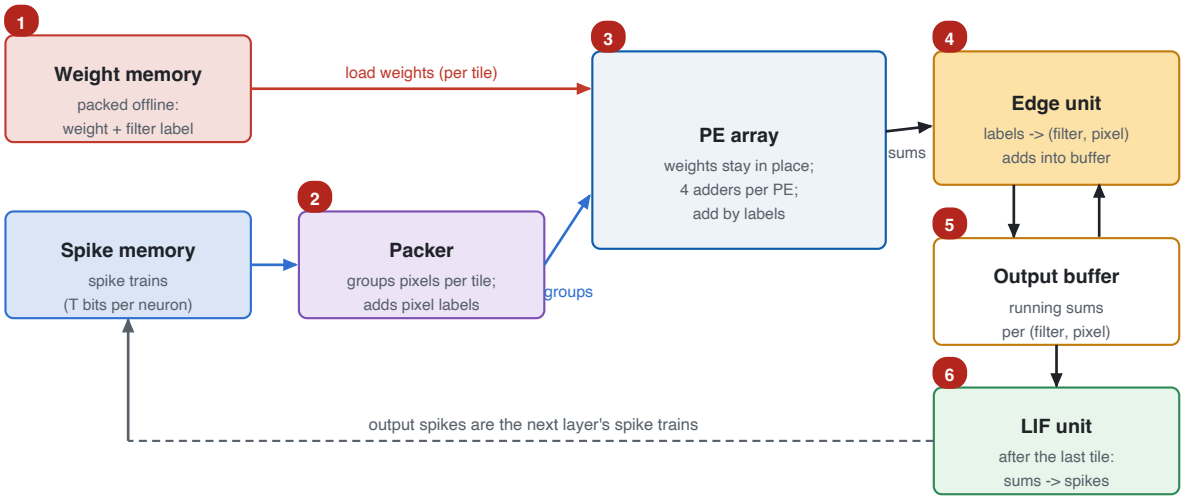


Figure 15. The blocks and the order data passes through them.

#	Block	What it does in the example
1	Weight memory	Holds the packed column (2IA, 3IB, 1IB, 1IA from top to bottom), prepared offline; loads it into the PEs.
2	Packer	Reads the spike trains of a tile, forms groups G1 and G2, and attaches a pixel label to each bit.
3	PE array	Each PE keeps one weight and its filter label; 4 adders add the weight into the running sum chosen by (filter label, bit's pixel label, timestep).
4	Edge unit	Takes each finished (or partial) running sum and its two labels, finds output[filter][pixel].
5	Output buffer	Adds the running sums of successive tiles together.
6	LIF unit	After the last tile, turns each running sum into an output spike train.

10. Why weight-stationary?

Another common arrangement is **output-stationary**: each PE keeps the running sums of its outputs and both weights and spike trains stream past. Packing works there too, but a PE then holds its outputs until the very last input of the layer, so a group must stay valid for **all** inputs of the layer. In a real layer two pixels almost always clash somewhere across hundreds of inputs. In weight-stationary, a group only has to be valid within one tile, and the next tile can regroup. Measured on ResNet18 (T = 8): grouping per tile of 4 inputs leaves 29% of the pixel steps, while grouping that must hold for the whole layer leaves 67%. The price of weight-stationary is the output buffer that adds partial sums between tiles (Section 7).

11. How this differs from earlier designs

	What is merged	Clash checked at	Weights sparse?	Timesteps	Matching logic in PEs
PTB (StSAP)	2 inputs that feed the same output	time window	no	partly parallel (windows)	no
Mentha (ANN)	rows / columns of a matrix	input (any non-zero)	yes	n/a (not SNN)	tag compare
LoAS	nothing; matches non-zeros instead	n/a	yes	all at once	yes (large)
Ours	filters (offline) and up to 4 pixels that read the same input (at run time)	single (input, timestep) bit	yes	all at once	no

PTB's StSAP is the closest idea: it also uses gaps in time to merge work. But it merges two **inputs** (which need two different weights), checks clashes per time window, and only pairs. We merge **outputs** that share a weight, check single timesteps, and allow groups larger than two, inside a PE that handles all timesteps at once and next to weight packing.

12. What it costs, and what is still open

- **Labels:** a filter label on every weight and a pixel label on every spike bit (1-2 bits each).

- **Running sums travelling together:** (filters per column) x (pixels per group) x T values per group; this is the main hardware cost and the reason group sizes are capped at 2-4.
- **Edge unit and output buffer:** one add per non-zero running sum per tile.
- **Packer:** forms groups at run time; it must keep up with the array.
- **Measured so far:** spike packing saves 1.1-1.4x on real data; compared with Prosperity, the design is faster only when weights are pruned beyond roughly 90%, and 2.5-5x faster at 98%.
- **Open:** group size (2 or 4), tile size, an energy model, and a comparison with LoAS at equal area.

13. Optimizations

13.1 The running sums carry no labels

Section 6 wrote every running sum as "(A, p0) [2,0,1,1]". The pair (A, p0) is correct, but it is **not stored** next to the sum. The labels are needed only inside the PE, to choose *which* running sum to add to. After that, the place of the sum in the bundle already says whose it is.

(a) The bundle is a row of numbered slots

A group carries F × G = 2 × 2 = 4 running sums down the column, always in the same order: **slot = 2 · filter label + member label**. The member label is local to the group: member 0 of G1 is p0, member 0 of G2 is p2.

slot e	filter label = e / 2	member label = e % 2	member in G1	member in G2
0	0 (A)	0	p0	p2
1	0 (A)	1	p1	p3
2	1 (B)	0	p0	p2
3	1 (B)	1	p1	p3

(b) Inside the PE, the two labels pick the slot

The PE computes the slot number from its own filter label and the label on each spike bit; it reads that slot, adds its weight, and writes it back. Nothing is searched: the number is known the moment the bits arrive.

PE k2: weight 1, filter label 1 (B)	lane t0	lane t1	lane t2	lane t3
spike bit of G1	0	1	1	1
member label	-	0 (p0)	1 (p1)	1 (p1)
slot = 2 · filter label + member label	no spike	2·1 + 0 = **2** → +1	2·1 + 1 = **3** → +1	2·1 + 1 = **3** → +1

Same cycle as Figure 10: +1 to slot 2 (B, p0) at t1 and to slot 3 (B, p1) at t2 and t3.

(c) What leaves the bottom: bare sums, one touch bit each, and one small sideband per group

G1 leaves the column

slot 0	[2,0,1,1]	touch 1
slot 1	[0,0,0,0]	touch 0
slot 2	[0,1,0,0]	touch 1
slot 3	[0,0,4,1]	touch 1

sideband (once per group): members [p0, p1]
column filter list [A, B] (fixed for the pass)

G2 leaves the column

slot 0	[1,2,0,0]	touch 1
slot 1	[0,1,2,2]	touch 1
slot 2	[0,0,3,3]	touch 1
slot 3	[1,0,0,0]	touch 1

sideband (once per group): members [p2, p3]
column filter list [A, B] (fixed for the pass)

(d) The unit below the array turns slot numbers back into (filter, pixel)

For slot e: filter = filter list[e / 2], pixel = members[e % 2]. Since e is fixed for every slot, this is plain wiring, not arithmetic (in the RTL: `{fid[e / G], pid[e % G], sums}`).

G1 slot	bare sums	list[e/2]	members[e%2]	write to
0	[2,0,1,1]	A	p0	output[A][p0]
1	[0,0,0,0]	A	p1	skip (touch 0)
2	[0,1,0,0]	B	p0	output[B][p0]
3	[0,0,4,1]	B	p1	output[B][p1]

G2 slot	bare sums	list[e/2]	members[e%2]	write to
0	[1,2,0,0]	A	p2	output[A][p2]
1	[0,1,2,2]	A	p3	output[A][p3]
2	[0,0,3,3]	B	p2	output[B][p2]
3	[1,0,0,0]	B	p3	output[B][p3]

	p0	p1	p2	p3
A	[2,0,1,1]	[0,0,0,0]	[1,2,0,0]	[0,1,2,2]
B	[0,1,0,0]	[0,0,4,1]	[0,0,3,3]	[1,0,0,0]

Output after both groups: identical to the GEMM (checked for every entry). Slot 1 of G1, (A, p1), was never touched, so nothing is written and output[A][p1] stays [0,0,0,0].

> **Rule.** A label is needed to *choose* a place; once a value sits in a fixed place, the place is its label. The weights and the spike bits carry labels because a PE cannot tell their filter or pixel from where they are. The running sums sit in fixed slots, so they carry none; the only extra data is the group's member list, sent once per group beside the column.

(e) What this saves in the real design (4 × 4 array, 16 slots of T = 8 sums)

	flip-flops	where
Fixed slots + ID sideband (ours)	**1,144**	one 52-bit ID record per group, delayed R + c cycles beside each column (4 + 5 + 6 + 7 stages)
13-bit (filter, pixel) address stored on every slot	3,328	16 slots × 13 b in the bundle register of each of the 16 PEs

Flip-flop counts from the RTL widths; about 2.9× fewer, and no comparator in any PE.